Lab 9

Introduction:

CPU scheduling is a fundamental function of an operating system that determines the order in which processes access the CPU. Since multiple processes compete for processor time, a scheduling algorithm improves system responsiveness, utilization, and fairness. In this exercise, three scheduling algorithms were implemented and tested:

- **First-Come, First-Served (FCFS)** – Non-preemptive, order based on arrival time.
- **Priority Scheduling (Non-Preemptive)** – Chooses the process with the highest priority (lowest number = highest priority).
- **Round Robin (RR)** – Preemptive time-shared scheduling with a fixed time quantum.

The purpose of this task is to compare these schedulers based on waiting time and turnaround time.

```
class Process:
    def __init__(self, pid, arrival_time, burst_time, priority=0):
        self.pid = pid
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.priority = priority

        # These will be calculated later
        self.completion_time = 0
        self.turnaround_time = 0
        self.waiting_time = 0
        self.start_time = 0

def print_results(title, processes, avg_turnaround, avg_waiting):
    """Prints the final results in a formatted table."""
    processes.sort(key=lambda x: x.pid)
    print(f"\n--- {title} Scheduling Results ---")
    print("=" * 70)
    print("PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting")
    print("-" * 70)
    for p in processes:
        print(f"{p.pid:^3} | {p.arrival_time:^7} | {p.burst_time:^5} | {p.priority:^8} | "
        f"{p.completion_time:^10} | {p.turnaround_time:^10} | {p.waiting_time:^7}")
    print("-" * 70)
    print(f"Average Turnaround Time: {avg_turnaround:.2f}")
    print(f"Average Waiting Time: {avg_waiting:.2f}")
    print("=" * 70)

def calculate_metrics(processes):
    """Calculates turnaround and waiting time for a list of completed processes."""
    total_turnaround_time = 0
    total_waiting_time = 0
    for p in processes:
        p.turnaround_time = p.completion_time - p.arrival_time
        p.waiting_time = p.turnaround_time - p.burst_time
        total_turnaround_time += p.turnaround_time
        total_waiting_time += p.waiting_time
    avg_turnaround_time = total_turnaround_time / len(processes)
    avg_waiting_time = total_waiting_time / len(processes)
    return avg_turnaround_time, avg_waiting_time
```

Priority Scheduling:

```
def priority_schedule(processes):
    processes = sorted(processes, key=lambda p: p.arrival_time)
    time = 0
    completed = []
    ready_queue = []
    idx = 0
    n = len(processes)

    while len(completed) < n:
        while idx < n and processes[idx].arrival_time <= time:
            ready_queue.append(processes[idx])
            idx += 1
        if not ready_queue:
            time = processes[idx].arrival_time
            continue
        ready_queue.sort(key=lambda p: (p.priority, p.arrival_time))
        current = ready_queue.pop(0)

        current.start_time = time
        time += current.burst_time
        current.completion_time = time

        completed.append(current)

    avg_tt, avg_wt = calculate_metrics(completed)
    print_results("Priority Scheduling (Non-Preemptive)", completed, avg_tt, avg_wt)
    return avg_tt, avg_wt
```

Round Robin (RR):

```
from collections import deque

def rr_schedule(processes, time_quantum):
    processes.sort(key=lambda p: p.arrival_time)
    remaining = {p.pid: p.burst_time for p in processes}
    ready = deque()
    time = 0
    i = 0
    completed = []

    while i < len(processes) or ready:
        while i < len(processes) and processes[i].arrival_time <= time:
            ready.append(processes[i])
            i += 1
        if not ready:
            time = processes[i].arrival_time
            continue
        current = ready.popleft()
        pid = current.pid
        run_time = min(time_quantum, remaining[pid])
        remaining[pid] -= run_time
        time += run_time
        while i < len(processes) and processes[i].arrival_time <= time:
            ready.append(processes[i])
            i += 1
        if remaining[pid] == 0:
            current.completion_time = time
            completed.append(current)
        else:
            ready.append(current)

    avg_tat, avg_wait = calculate_metrics(completed)
    print_results("Round Robin", completed, avg_tat, avg_wait)
    return avg_tat, avg_wait
```

First-Come, First-Served (FCFS):

```python
def fcfs_schedule(processes):
# Sort processes based on arrival time
    processes_sorted = sorted(processes, key=lambda p: p.arrival_time)
    current_time = 0
    for p in processes_sorted:
        if current_time < p.arrival_time:
            current_time = p.arrival_time
        p.start_time = current_time
        p.completion_time = current_time + p.burst_time
        current_time = p.completion_time
    avg_tt, avg_wt = calculate_metrics(processes_sorted)
    print_results("First-Come, First-Served (FCFS)", processes_sorted, avg_tt, avg_wt)
    return avg_tt, avg_wt

# if __name__ == "__main__":
# #     workload = [
# #         Process(1, 0, 8, 3),
# #         Process(2, 1, 4, 1),
# #         Process(3, 2, 9, 4),
# #         Process(4, 3, 5, 2),
# #     ]

# #     print("--- Starting Priority Scheduling Simulation ---")
# #     priority_schedule(workload)

# #     processes = [
# #     {"pid": 1, "arrival_time": 0, "burst_time": 5, "priority": 2},
# #     {"pid": 2, "arrival_time": 1, "burst_time": 3, "priority": 1},
# #     {"pid": 3, "arrival_time": 2, "burst_time": 8, "priority": 3}
# # ]

# # # Convert dict → Process objects
# # processes = [Process(p["pid"], p["arrival_time"], p["burst_time"], p["priority"]) for p in processes]

# # rr_schedule(processes, time_quantum=2)

# # workload_fcfs = [
# # Process(1, 0, 8, 3),
# # Process(2, 1, 4, 1),
# # Process(3, 2, 9, 4),
# # Process(4, 3, 5, 2),
# # ]
# # print("--- Starting FCFS Simulation ---")
# # # Run the FCFS scheduler with a deep copy of the workload
# # fcfs_schedule((workload_fcfs))

if __name__ == "__main__":
    # Create a test workload of at least 5 processes
    workload = [
        Process(1, 0, 7, 2),
        Process(2, 1, 4, 1),
        Process(3, 2, 9, 3),
        Process(4, 3, 5, 2),
        Process(5, 6, 2, 1)
    ]

    import copy

print("\n===== Running FCFS =====")
fcfs_tt, fcfs_wt = fcfs_schedule(copy.deepcopy(workload))

print("\n===== Running Priority Scheduling =====")
priority_tt, priority_wt = priority_schedule(copy.deepcopy(workload))

print("\n===== Running Round Robin (Time Quantum = 2) =====")
rr_tt, rr_wt = rr_schedule(copy.deepcopy(workload), time_quantum=2)

print("\n========== Summary Comparison ==========")
print("Algorithm         | Avg Turnaround | Avg Waiting")
print("-----------------------------------------------")
print(f"FCFS              | {fcfs_tt:14.2f} | {fcfs_wt:10.2f}")
print(f"Priority          | {priority_tt:14.2f} | {priority_wt:10.2f}")
print(f"Round Robin (Q=2)| {rr_tt:14.2f} | {rr_wt:10.2f}")
print("-----------------------------------------------")
```

```
===== Running FCFS =====

--- First-Come, First-Served (FCFS) Scheduling Results ---
======================================================================
PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting
----------------------------------------------------------------------
 1  |    0    |   7   |    2     |     7      |     7      |    0
 2  |    1    |   4   |    1     |     11     |     10     |    6
 3  |    2    |   9   |    3     |     20     |     18     |    9
 4  |    3    |   5   |    2     |     25     |     22     |   17
 5  |    6    |   2   |    1     |     27     |     21     |   19
----------------------------------------------------------------------
Average Turnaround Time: 15.60
Average Waiting Time: 10.20
======================================================================
```

```
===== Running Priority Scheduling =====

--- Priority Scheduling (Non-Preemptive) Scheduling Results ---
================================================================================
PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting
--------------------------------------------------------------------------------
 1  |    0    |   7   |    2     |     7      |     7      |    0
 2  |    1    |   4   |    1     |    11      |    10      |    6
 3  |    2    |   9   |    3     |    27      |    25      |   16
 4  |    3    |   5   |    2     |    18      |    15      |   10
 5  |    6    |   2   |    1     |    13      |     7      |    5
--------------------------------------------------------------------------------
Average Turnaround Time: 12.80
Average Waiting Time: 7.40
================================================================================

===== Running Round Robin (Time Quantum = 2) =====

--- Round Robin Scheduling Results ---
================================================================================
PID | Arrival | Burst | Priority | Completion | Turnaround | Waiting
--------------------------------------------------------------------------------
 1  |    0    |   7   |    2     |     23     |     23     |   16
 2  |    1    |   4   |    1     |     12     |     11     |    7
 3  |    2    |   9   |    3     |     27     |     25     |   16
 4  |    3    |   5   |    2     |     24     |     21     |   16
 5  |    6    |   2   |    1     |     14     |      8     |    6
--------------------------------------------------------------------------------
Average Turnaround Time: 17.60
Average Waiting Time: 12.20
================================================================================

========= Summary Comparison =========
Algorithm         | Avg Turnaround | Avg Waiting
-----------------------------------------------
FCFS              |          15.60 |       10.20
Priority          |          12.80 |        7.40
Round Robin (Q=2)|          17.60 |       12.20
-----------------------------------------------
```

Which scheduler performed best?

The Priority Scheduling algorithm performed the best for this workload.
It achieved the lowest average waiting time (7.40) and lowest turnaround time (12.80) because it always executes the highest-priority (lowest-numbered) process next, reducing delays for short and important processes.

Why choose Round Robin even with higher waiting time?

Round Robin is preferred in time-sharing and interactive systems, where fairness and responsiveness are more important than average waiting time.
Even if waiting time is higher, RR ensures:

- Every process gets CPU time regularly
- No starvation
- Better user experience in multitasking systems

This makes RR ideal for operating systems used by multiple active users.

Drawback of Priority Scheduling

The main drawback is starvation.
Lower-priority processes may never get CPU time if higher-priority ones keep arriving.
A long process with low priority may stay in the ready queue indefinitely.

Conclusion

This experiment demonstrated how different scheduling algorithms affect process completion time. Priority Scheduling gave the best performance for the given workload, while FCFS produced moderate results but suffered from long waiting times for later processes. Round Robin ensured fairness but had the highest waiting and turnaround times due to frequent context switching.
Choosing a scheduler depends on the system's goals—whether fairness, response time, or overall throughput is more important.